

SBT-DF203 — Basic Networking Skills for Digital Forensics

LAB 6 — FIREWALL TRAFFIC CONTROL AND FORENSIC VERIFICATION

Prepared by: Oluyede Tobi Steven

Registration Number: 202513549PF

Course: SBT-DF203 — Basic Networking Skills for Digital Forensics

Lab: Lab 6 — Firewall Traffic Control and Forensic Verification

Date: 18 September 2026

Instructor: Aminu Idris, AMCPN

Environment: Isolated Virtual Laboratory

1. Executive Summary

This laboratory exercise investigated the operation of a Linux host-based firewall and its effect on HTTP traffic between an authorized laboratory client and a protected Linux server.

The exercise began by preserving the original firewall configuration using iptables-save and recording the active firewall rules, network configuration, and Apache HTTP listener. A successful HTTP connection was then established from an authorized client to provide a baseline for comparison.

An allowed HTTP session was captured using TShark and preserved as PCAPNG evidence. A narrowly scoped iptables INPUT rule was subsequently inserted to silently DROP TCP port 80 traffic originating from the assigned blocked laboratory client. The rule was verified and its packet and byte counters were monitored.

A second packet capture was collected while the blocked client attempted to access the Apache webpage. The resulting traffic was compared with the successful baseline. The analysis focused on TCP connection establishment, HTTP request and response visibility, retransmissions, client timeout behaviour, and firewall counter changes.

Finally, the exact DROP rule was removed and the firewall state was checked. HTTP access from the previously blocked client was tested again to confirm restoration of normal service.

The exercise demonstrated how firewall configuration evidence, packet captures, TCP behaviour, application output, and firewall counters can be correlated to support a forensic interpretation of network access control.

2. Lab Scenario

The purpose of the exercise was to validate whether a Linux host-based firewall could block HTTP traffic from one authorized laboratory client while allowing HTTP traffic from another authorized client.

The investigation followed four main states:

1. Original firewall configuration and successful HTTP baseline.
2. Allowed HTTP traffic captured as reference evidence.
3. Source-specific TCP/80 DROP rule applied and blocked behaviour captured.
4. Firewall rule removed and HTTP access restored.

All testing was performed within an authorized isolated laboratory environment.

3. Objectives

The objectives of this laboratory were to:

- Explain the difference between host-based and network-based firewalls.
 - Understand the function of the INPUT, OUTPUT, and FORWARD iptables chains.
 - Preserve and document the original firewall configuration before modification.
 - Create a narrowly scoped source-specific HTTP blocking rule.
 - Verify that the firewall rule was correctly installed.
 - Capture an allowed HTTP session.
 - Capture and analyze blocked HTTP connection attempts.
 - Correlate firewall packet/byte counters with packet-capture evidence.
 - Identify TCP retransmissions and timeout behaviour associated with a DROP action.
 - Explain the difference between DROP and REJECT.
 - Safely remove the firewall rule.
 - Confirm restoration of HTTP access.
 - Preserve hashes and supporting evidence for forensic verification.
-

4. Scope, Authorization and Safety

The practical work was restricted to the approved ICDFA laboratory environment.

No production, public, campus, office, third-party or unauthorized systems were targeted. The firewall modification was limited to a source-specific TCP port 80 rule.

The original firewall ruleset was exported before modification. Packet captures were preserved as original evidence, while analysis was performed using appropriate working copies where required.

The firewall rule was removed after testing to restore the system to its previous operational state.

5. Host-Based and Network-Based Firewalls

A **host-based firewall** operates directly on an individual computer and controls traffic entering or leaving that host. In this laboratory, iptables was used as the host-based firewall mechanism on the Linux server.

A **network-based firewall** controls traffic between networks or network segments. It is commonly positioned at a network boundary and can enforce security policies for multiple hosts.

The primary distinction is therefore the location and scope of enforcement:

Firewall Type	Main Function
Host-based firewall	Controls traffic associated with an individual host
Network-based firewall	Controls traffic between networks or network segments

6. iptables Chain Interpretation

The laboratory used the INPUT chain because the HTTP traffic was destined for the protected Linux server.

INPUT

The INPUT chain processes packets destined for the local host.

OUTPUT

The OUTPUT chain processes packets generated by the local host and leaving it.

FORWARD

The FORWARD chain processes packets being routed through the host rather than packets destined for the host itself.

In this investigation, the relevant traffic path was:

Laboratory Client → Linux Server → Apache HTTP Service

Because the HTTP packets were destined for the Linux server itself, the source-specific blocking rule was placed in the INPUT chain.

7. Laboratory Environment

The practical environment consisted of:

- Linux server VM running Apache.
 - Authorized HTTP client VM.
 - Separate assigned blocked client VM.
 - Isolated virtual network.
 - iptables for firewall control.
 - TShark/Wireshark for packet capture and analysis.
 - curl for HTTP testing.
-

8. Evidence Directory Preparation

A dedicated laboratory directory was created:

```
mkdir -p ~/SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab6
pwd
find . -maxdepth 1 -type d -print
```

~/SBT-DF203-Lab6/

```
├── evidence/
├── working/
├── exported/
├── reports/
├── screenshots/
└── scripts/
```

```
(kali㉿kali)-[~/icdfaDF/SBT-DF203-Lab5]
$ cd ..
(kali㉿kali)-[~/icdfaDF]
$ mkdir -p SBT-DF203-Lab6/{evidence,working,exported,reports,screenshots,scripts}
cd SBT-DF203-Lab6
pwd
find . -maxdepth 1 -type d -print
/home/kali/icdfaDF/SBT-DF203-Lab6
./screenshots
./working
./evidence
./reports
./exported
./scripts
(kali㉿kali)-[~/icdfaDF/SBT-DF203-Lab6]
```

The purpose of the structure was to separate original evidence, analysis copies, exported command output, screenshots and report materials.

Downloading evidence

```
sudo apt update
sudo apt install -y apache2 curl iptables tshark wireshark
sudo systemctl enable --now apache2
printf '<!DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>\n' | sudo tee /var/www/html/firewall_lab.html
```

```
(kali㉿kali)-[~/icdfaDF/SBT-DF203-Lab6]
$ sudo apt update
$ sudo apt install -y apache2 curl iptables tshark wireshark
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.9 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [41.8 kB]
Fetched 77.0 MB in 31s (2,489 kB/s)
```

```
(kali@kali)-[~/icdfaDF/STB-DF203-Lab6]
$ printf '<DOCTYPE html>\n<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: steven </p></body></html>\n' | sudo tee /var/www/html/firewall_lab.html
[sudo] password for kali:
<DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: steven </p></body></html>
(kali@kali)-[~/icdfaDF/STB-DF203-Lab6]
$
```

Original Firewall State

Before modifying the firewall, the current ruleset was exported using:

```
sudo iptables-save | tee reports/iptables_before.rules
```

The active firewall rules and counters were recorded using:

```
sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
```

The SHA-256 hash of the exported ruleset was calculated:

```
sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt
```

The original ruleset was preserved before any firewall modification was made.

```
(kali@kali)-[~/icdfaDF/STB-DF203-Lab6]
$ sudo iptables-save | tee reports/iptables_before.rules
sudo iptables -L -n -v --line-numbers | tee reports/iptables_before.txt
sha256sum reports/iptables_before.rules | tee reports/iptables_before_sha256.txt

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source               destination
e3b0c44298fc149afb4c8996fb92427ae41e4649b934ca495991b7852b855  reports/iptables_before.rules
(kali@kali)-[~/icdfaDF/STB-DF203-Lab6]
$
```

Network and Apache Baseline

The server's active network configuration was recorded using:

```
ip -br address | tee reports/server_interfaces.txt
```

The routing table was recorded using:

```
ip route | tee reports/server_routes.txt
```

The Apache listener was verified using:

```
sudo ss -lntp | grep ':80' | tee reports/apache_listener.txt
```

The output confirmed that the web service was listening on TCP port 80.

```
(kali@kali)~[~/icdafaDF/SBT-DF203-Lab6]
$ ip -br address | tee reports/server_interfaces.txt
ip route | tee reports/server_routes.txt
sudo ss -lntp | grep ':80' | tee reports/apache_listener.txt
lo        UNKNOWN      127.0.0.1/8  ::1/128
eth0      UP              192.168.32.130/24  fe80::a75:8a8c:87c9:e448/64
default via 192.168.32.2 dev eth0 proto dhcp src 192.168.32.130 metric 100
192.168.32.0/24 dev eth0 proto kernel scope link src 192.168.32.130 metric 100
LISTEN 0      511          *:80          *:80    users:((("apache2",pid=1278200,fd=4),("apache2",pid=1278199,fd=4),("apache2",pid=1278198,fd=4),("apach
78196,fd=4),("apache2",pid=1278180,fd=4))
```

Baseline HTTP Test

From the authorized client, the following command was used:

```
curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html
```

```
[attacker@parrot]~[~]
$ curl -v --connect-timeout 5 http://192.168.32.130/firewall_lab.html
* Trying 192.168.32.130:80...
* Connected to 192.168.32.130 (192.168.32.130) port 80
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.32.130
> User-Agent: curl/8.11.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 16:51:35 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 16:35:01 GMT
< ETag: "77-65bc47b5970c2"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>
* Connection #0 to host 192.168.32.130 left intact
```

Field	Value
Server IP	192.168.32.130

Server interface	eth0
Loopback IP	127.0.0.1
Network	192.168.32.0/24
Default gateway	192.168.32.2
Apache listener	*:80
Apache status	Listening
Apache process	apache2

Allowed HTTP Packet Capture

The allowed HTTP traffic was captured using TShark:

```
sudo tshark -i IFACE \  
-f 'host CLIENT_IP and tcp port 80' \  
-a duration:30 \  
-w evidence/http_allowed.pcapng
```

The authorized client generated an HTTP request while the capture was active.

```
(kali㉿kali)-[~/icdfaDF/GBT-DF203-Lab6]  
--$ sudo tshark -i eth0 -f 'host 192.168.32.142 and tcp port 80' -a duration:30 -w /tmp/http_allowed.pcapng  
Running as user "root" and group "root". This could be dangerous.  
Capturing on 'eth0'  
10
```

The resulting evidence file was:

evidence/http_allowed.pcapng

The SHA-256 hash was calculated:

```
Sudo sha256sum evidence/http_allowed.pcapng \  
| tee reports/http_allowed_sha256.txt
```

Allowed PCAP Hash

7609f47d20a3cfca9b7d32978aa07936253c20af7f672c1a189407e5572b2a5c

```
(kali㉿kali)-[~/icdfaDF/GBT-DF203-Lab6]  
$ sudo sha256sum evidence/http_allowed.pcapng | tee reports/http_allowed_sha256.txt  
7609f47d20a3cfca9b7d32978aa07936253c20af7f672c1a189407e5572b2a5c  evidence/http_allowed.pcapng  
  
(kali㉿kali)-[~/icdfaDF/GBT-DF203-Lab6]  
$
```

Allowed Traffic Analysis

The allowed capture was examined for TCP and HTTP activity.

A normal successful TCP connection is expected to contain the following sequence:

Client → Server	SYN
Server → Client	SYN-ACK
Client → Server	ACK
Client → Server	HTTP GET
Server → Client	HTTP Response

The capture showed:

- **Client SYN:** [YES/NO]
- **Server SYN-ACK:** [YES/NO]
- **TCP handshake completed:** [YES/NO]
- **HTTP GET visible:** [YES/NO]
- **HTTP response visible:** [YES/NO]
- **HTTP response code:** [INSERT CODE]

The presence of the TCP handshake followed by an HTTP request and response is consistent with successful communication between the client and Apache server.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.32.142	192.168.32.130	TCP	74	36426 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2530606251 TSecr=0 WS=128
2	0.000036838	192.168.32.130	192.168.32.142	TCP	74	80 → 36426 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=1365214059 TSecr=25306...
3	0.000420923	192.168.32.142	192.168.32.130	TCP	66	36426 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=2530606252 TSecr=1365214059
4	0.000719850	192.168.32.142	192.168.32.130	HTTP	161	GET /firewall_lab.html HTTP/1.1
5	0.000820161	192.168.32.130	192.168.32.142	TCP	66	80 → 36426 [ACK] Seq=1 Ack=96 Win=65536 Len=0 TSval=1365214059 TSecr=2530606253
6	0.011142254	192.168.32.130	192.168.32.142	HTTP	436	HTTP/1.1 200 OK (text/html)
7	0.011537421	192.168.32.142	192.168.32.130	TCP	66	36426 → 80 [ACK] Seq=96 Ack=371 Win=64128 Len=0 TSval=2530606266 TSecr=1365214070
8	0.013974681	192.168.32.142	192.168.32.130	TCP	66	36426 → 80 [FIN, ACK] Seq=96 Ack=371 Win=64128 Len=0 TSval=2530606266 TSecr=1365214070
9	0.014165983	192.168.32.130	192.168.32.142	TCP	66	80 → 36426 [FIN, ACK] Seq=371 Ack=97 Win=65536 Len=0 TSval=1365214073 TSecr=2530606266
10	0.014500952	192.168.32.142	192.168.32.130	TCP	66	36426 → 80 [ACK] Seq=97 Ack=372 Win=64128 Len=0 TSval=2530606266 TSecr=1365214073

Firewall Rule Implementation

The assigned blocked client address was recorded as:

BLOCKED_CLIENT_IP = [INSERT BLOCKED CLIENT IP]

The blocking rule was inserted at the top of the INPUT chain:

```
sudo iptables -I INPUT 1 -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
```

```
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt
```

```
(kali@kali)-[~/icdfaDF/STB-DF203-Lab6]
$ sudo iptables -I INPUT 1 -s 192.168.32.142 -p tcp --dport 80 -j DROP

(kali@kali)-[~/icdfaDF/STB-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_add.txt

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source       destination
1          0      0 DROP      tcp  --  *      *       192.168.32.142  0.0.0.0/0      tcp dpt:80

(kali@kali)-[~/icdfaDF/STB-DF203-Lab6]
$
```

Rule Verification

The resulting INPUT chain was recorded:

```
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
```

```
printf 'Rule verified present.\n' | tee reports/rule_verification.txt
```

```
(kali@kali)-[~/icdfaDF/GBT-DF203-Lab6]
$ sudo iptables -C INPUT -s "192.168.32.142" -p tcp --dport 80 -j DROP
printf 'Rule verified present.\n' | tee reports/rule_verification.txt
```

```
Rule verified present.
```

```
(kali@kali)-[~/icdfaDF/GBT-DF203-Lab6]
$
```

Blocked HTTP Traffic Capture

A second capture was performed while the blocking rule was active:

```
sudo tshark -i IFACE \
-f 'host BLOCKED_CLIENT_IP and tcp port 80' \
-a duration:35 \
-w evidence/http_blocked.pcapng
```

```
(kali@kali)-[~/icdfaDF/GBT-DF203-Lab6]
$ sudo tshark -i eth0 -f 'host 192.168.32.142 and tcp port 80' -a duration:30 -w /tmp/http_blocked.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
5

(kali@kali)-[~/icdfaDF/GBT-DF203-Lab6]
$ sudo mv /tmp/http_blocked.pcapng evidence/http_blocked.pcapng

(kali@kali)-[~/icdfaDF/GBT-DF203-Lab6]
```

The blocked client attempted to access:

```
curl -v --connect-timeout 10 \
http://SERVER_IP/firewall_lab.html
```

```

[attacker@parrot]-[~]
$ curl -v --connect-timeout 5 http://192.168.32.130/firewall_lab.html
* Trying 192.168.32.130:80...
* Connection timed out after 5002 milliseconds
* closing connection #0
curl: (28) Connection timed out after 5002 milliseconds
[X]-[attacker@parrot]-[~]
$

```

The resulting capture was preserved as:

evidence/http_blocked.pcapng

Its SHA-256 hash was calculated:

```

sha256sum evidence/http_blocked.pcapng \
| tee reports/http_blocked_sha256.txt

```

```

(kali@kali)-[~/icdfADF/STB-DF203-Lab6]
$ sudo sha256sum evidence/http_blocked.pcapng | tee reports/http_blocked_sha256.txt
75c49f4600bd524b74657635b1825c3c80de9c68b6f6786d2c2f74a5cd2546eb  evidence/http_blocked.pcapng
(kali@kali)-[~/icdfADF/STB-DF203-Lab6]
$

```

Blocked PCAP Hash

75c49f4600bd524b74657635b1825c3c80de9c68b6f6786d2c2f74a5cd2546eb
evidence/http_blocked.pcapng

Blocked Client Behaviour

The blocked client produced the following observed result:

[INSERT ACTUAL CURL OUTPUT / TIMEOUT RESULT]

The important forensic observation was whether the connection remained unanswered until the client timed out.

A DROP action silently discards matching packets. Because the firewall does not send a TCP response to the client, the client may continue retransmitting SYN packets while waiting for a response.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.32.142	192.168.32.130	TCP	74	40218 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2533950349 TSecr=0 WS=128
2	1.027287498	192.168.32.142	192.168.32.130	TCP	74	[TCP Retransmission] 40218 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2533951375 TSecr=0 WS=128
3	2.048437976	192.168.32.142	192.168.32.130	TCP	74	[TCP Retransmission] 40218 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2533952389 TSecr=0 WS=128
4	3.054289902	192.168.32.142	192.168.32.130	TCP	74	[TCP Retransmission] 40218 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2533953403 TSecr=0 WS=128
5	4.067067486	192.168.32.142	192.168.32.130	TCP	74	[TCP Retransmission] 40218 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2533954415 TSecr=0 WS=128

Firewall Counter Correlation

After the blocked test, the INPUT chain was recorded:

```
sudo iptables -L INPUT -n -v --line-numbers \
```

```
| tee reports/iptables_after_test.txt
```

```
(kali@kali)-[~/icdfaDF/SBT-DF203-Lab6]
$ sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_after_test.txt

Chain INPUT (policy ACCEPT 107 packets, 26388 bytes)
num  pkts bytes target    prot opt in     out     source               destination
1      5  300 DROP          tcp  --  *      *        192.168.32.142      0.0.0.0/0            tcp dpt:80

(kali@kali)-[~/icdfaDF/SBT-DF203-Lab6]
$
```

Capture Comparison Command

The two PCAPNG files were examined using:

```
for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
  echo "=====$PCAP===="
  tshark -r "$PCAP" -Y 'tcp.flags.syn==1 || http.request || http.response ||
  tcp.analysis.retransmission' \
    -T fields -e frame.number -e frame.time_relative -e ip.src -e tcp.srcport -e ip.dst -e
  tcp.dstport \
    -e tcp.flags -e tcp.analysis.retransmission -e http.request.uri -e http.response.code
done | tee reports/allowed_vs_blocked.tsv
```

This output provided a structured basis for comparing connection establishment, HTTP activity and retransmissions.

```

(root@kali)-[/home/kali/icdfADF/STB-DF203-Lab6]
# for PCAP in evidence/http_allowed.pcapng evidence/http_blocked.pcapng; do
echo "==== $PCAP ===="
tshark -r "$PCAP" \
-Y 'tcp.flags.syn==1 || http.request || http.response || tcp.analysis.retransmission' \
-T fields \
-e frame.number \
-e frame.time_relative \
-e ip.src \
-e tcp.srcport \
-e ip.dst \
-e tcp.dstport \
-e tcp.flags \
-e tcp.analysis.retransmission \
-e http.request.uri \
-e http.response.code
done | tee reports/allowed_vs_blocked.tsv

==== evidence/http_allowed.pcapng ====
Running as user "root" and group "root". This could be dangerous.
1 0.000000000 192.168.32.142 36426 192.168.32.130 80 0x0002
2 0.000036838 192.168.32.130 80 192.168.32.142 36426 0x0012
4 0.000719850 192.168.32.142 36426 192.168.32.130 80 0x0018 /firewall_lab.html
6 0.01142254 192.168.32.130 80 192.168.32.142 36426 0x0018 /firewall_lab.html 200

==== evidence/http_blocked.pcapng ====
Running as user "root" and group "root". This could be dangerous.
1 0.000000000 192.168.32.142 40218 192.168.32.130 80 0x0002
2 1.027287498 192.168.32.142 40218 192.168.32.130 80 0x0002 1
3 2.040437976 192.168.32.142 40218 192.168.32.130 80 0x0002 1
4 3.054289902 192.168.32.142 40218 192.168.32.130 80 0x0002 1
5 4.067067486 192.168.32.142 40218 192.168.32.130 80 0x0002 1

```

DROP Versus REJECT

The DROP target silently discards matching packets. No explicit error response is generated by the firewall for the blocked connection.

As a result, a TCP client may continue waiting for a response and retransmit connection attempts before eventually reaching its timeout.

A REJECT action behaves differently because the firewall actively informs the sender that the traffic is not being accepted. Depending on the protocol and rejection option, this may involve an ICMP error or a TCP reset.

Indicator	Allowed Capture	Blocked Capture
Client SYN visible?	Yes — Frame 1	Yes — Frame 1

Server SYN-ACK visible?	Yes — Frame 2	No
Handshake completed?	Yes	No
HTTP GET visible?	Yes — Frame 4, /firewall_lab.html	No
HTTP response visible?	Yes — Frame 6, 200 OK	No
Retransmissions/timeouts	None shown	Yes — Frames 2–5 are repeated SYN retransmissions
iptables counter change	Not shown in this capture output; verify from your rule-counter evidence	Should increase when the DROP rule matches; record the actual before/after values from iptables -L INPUT -n -v
curl result	Successful HTTP retrieval	Failed/timed out while the DROP rule was active

Distinguishing Firewall Blocking from Server Failure

A timeout alone does not conclusively prove that a firewall rule caused the failure.

Other possible causes include:

- Apache being stopped.
- The server not listening on TCP/80.
- Incorrect routing.
- Network connectivity failure.

- Host or virtual-network configuration problems.
- Another firewall rule affecting the traffic.

For this reason, the investigation used multiple evidence sources.

The Apache listener was verified before the blocking rule:

```
sudo ss -lntp | grep ':80'
```

The successful baseline demonstrated that HTTP service was available before the firewall modification.

The blocked capture, firewall rule and counter changes then provided additional evidence for interpreting the blocked state.

This multi-source correlation is stronger than relying solely on the client's timeout message.

Firewall Rule Removal

After completing the blocked-traffic analysis, the exact firewall rule was removed using:

```
sudo iptables -D INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP
sudo iptables -C INPUT -s "$BLOCKED_CLIENT_IP" -p tcp --dport 80 -j DROP ||
echo 'Rule successfully removed'
sudo iptables -L INPUT -n -v --line-numbers | tee
reports/iptables_restored.txt
```

```
(kali@kali)-[~/icdfaDF/GBT-DF203-Lab6]
$ sudo iptables -D INPUT -s "192.168.32.142" -p tcp --dport 80 -j DROP
sudo iptables -C INPUT -s "192.168.32.142" -p tcp --dport 80 -j DROP || echo 'Rule successfully removed'
sudo iptables -L INPUT -n -v --line-numbers | tee reports/iptables_restored.txt

[sudo] password for kali:
iptables: Bad rule (does a matching rule exist in that chain?).
iptables: Bad rule (does a matching rule exist in that chain?).
Rule successfully removed
tee: reports/iptables_restored.txt: Permission denied
Chain INPUT (policy ACCEPT 365 packets, 80002 bytes)
num  pkts bytes target    prot opt in     out     source                     destination
```

Restoration Test

The previously blocked client was tested again using:

```
# Confirm access from the previously blocked client
# curl -v --connect-timeout 5 http://SERVER_IP/firewall_lab.html
```

```
[attacker@parrot]~$ curl -v --connect-timeout 5 http://192.168.32.130/firewall_lab.html
* Trying 192.168.32.130:80...
* Connected to 192.168.32.130 (192.168.32.130) port 80
* using HTTP/1.x
> GET /firewall_lab.html HTTP/1.1
> Host: 192.168.32.130
> User-Agent: curl/8.11.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Fri, 18 Sep 2026 18:09:17 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Fri, 18 Sep 2026 16:35:01 GMT
< ETag: "77-65bc47b5970c2"
< Accept-Ranges: bytes
< Content-Length: 119
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>ICDFA Network Forensics Firewall Lab</h1><p>Server: YOUR SERVER NAME</p></body></html>
* Connection #0 to host 192.168.32.130 left intact
[attacker@parrot]~$
```

Evidence Integrity

The following evidence files were preserved:SHA-256 hashing was used to provide integrity verification for the collected evidence.

The original evidence files should not be modified after hashing. Any analysis copies should be separately identified and, where required, hashed independently.

Answers to Forensic Interpretation Questions

Why does DROP commonly cause SYN retransmissions and a timeout?

A DROP action silently discards matching packets without sending a response to the client. If a TCP SYN is discarded, the client does not receive the expected SYN-ACK. It may therefore retransmit the SYN while waiting for the connection to be established. After the configured connection timeout is reached, the client reports a failure.

How would a REJECT rule differ in the packet capture and client output?

REJECT actively informs the sender that the connection is not permitted. Depending on the configured rejection method, the capture may contain a TCP reset or an ICMP error. The client will generally receive an immediate indication of failure rather than waiting through repeated unanswered attempts.

Why are firewall counters valuable corroborating evidence?

Firewall counters provide local evidence that packets matched a particular rule. If the packet and byte counters increase during the blocked test, this supports the interpretation that the source-specific rule processed matching traffic.

Counters should be correlated with packet captures and application behaviour because counters alone do not provide complete information about the connection.

What evidence would show that the web server was down rather than firewall-blocked?

Useful evidence would include:

- Apache not listening on TCP/80.
- Failure from multiple authorized clients.
- No successful baseline immediately before the test.
- Service-status output showing Apache stopped or failed.
- Packet captures showing traffic reaching the server without a corresponding service response.
- Firewall counters not increasing for the blocking rule.

A successful baseline followed by a source-specific firewall rule hit provides a different evidential picture.

What is the risk of deleting a rule by line number after other rules have changed?

Rule numbers can change when rules are inserted or deleted. Deleting by a line number after the ruleset has changed could therefore remove the wrong rule.

Using the exact rule specification with:

```
iptables -D INPUT -s SOURCE_IP -p tcp --dport 80 -j DROP
```

reduces this risk because the intended rule is identified by its properties rather than its current position.

Conclusion

This laboratory demonstrated the forensic verification of Linux host-based firewall traffic control.

The original firewall state was preserved before modification, and a successful HTTP baseline was established. An allowed packet capture documented the normal TCP and HTTP communication sequence.

A narrowly scoped source-specific TCP/80 DROP rule was then inserted into the INPUT chain and verified. The blocked client behaviour was captured and compared with the allowed session. Particular attention was given to the absence of expected server responses, retransmission behaviour, client timeout and increases in firewall packet and byte counters.

The exact rule was subsequently removed and HTTP access was tested again to verify restoration.

The exercise demonstrated the importance of correlating firewall configuration, rule counters, packet captures and application behaviour when conducting network forensic analysis. It also demonstrated why preserving the original ruleset and collecting integrity hashes are important components of a repeatable forensic workflow.

